



BÁO CÁO

BỘ LẬP LỊCH TÙY CHỈNH VÀ THUẬT TOÁN CẤP PHÁT TÀI NGUYÊN CHO



KUBERNETES

2425II_INT3310_1

QUẢN TRỊ MẠNG

NHÓM 1 - ĐỀ TÀI 8 - KUBERNETES



Nội dung chính



I.

Giới thiệu
Kubernetes &
Nhu cầu lập lịch

III.

Bộ lập lịch tùy chỉnh
(**custom scheduler**)

II.

Bộ lập lịch mặc định
(**kube-scheduler**)

IV.

Demo custom
scheduler



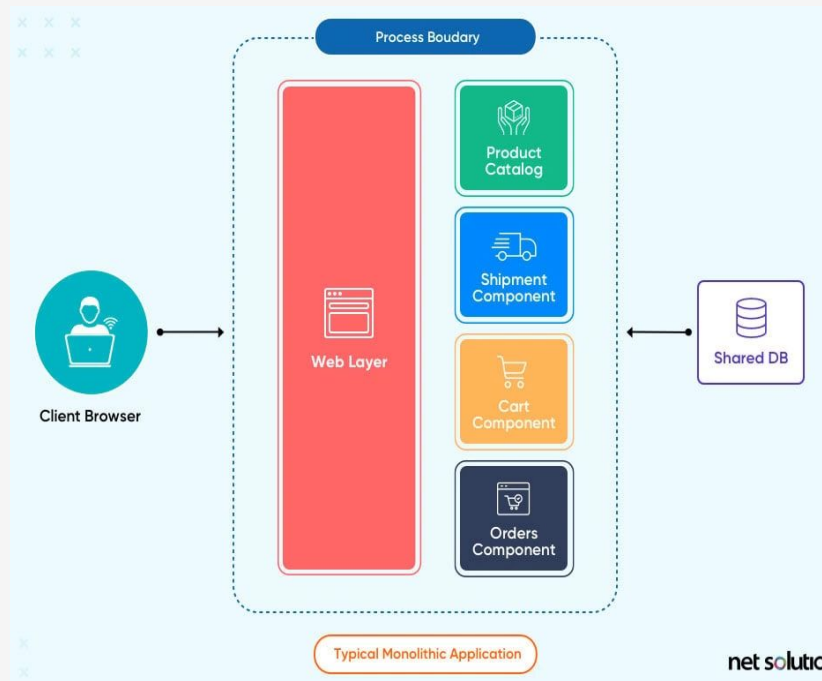
I.

Giới thiệu **Kubernetes** & Nhu cầu lập lịch



Kiến trúc Monolithic

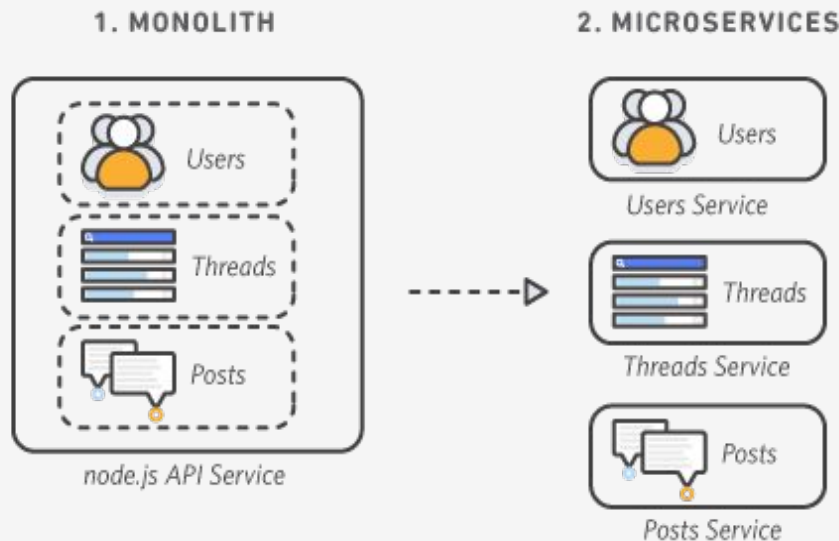
- Các thành phần bên trong **hộp nhất** thành một **hệ thống duy nhất**, có liên kết **phụ thuộc chặt chẽ**.
- **Ưu điểm:** Đơn giản
- **Nhược điểm:** khó mở rộng, phát triển



Kiến trúc Microservices

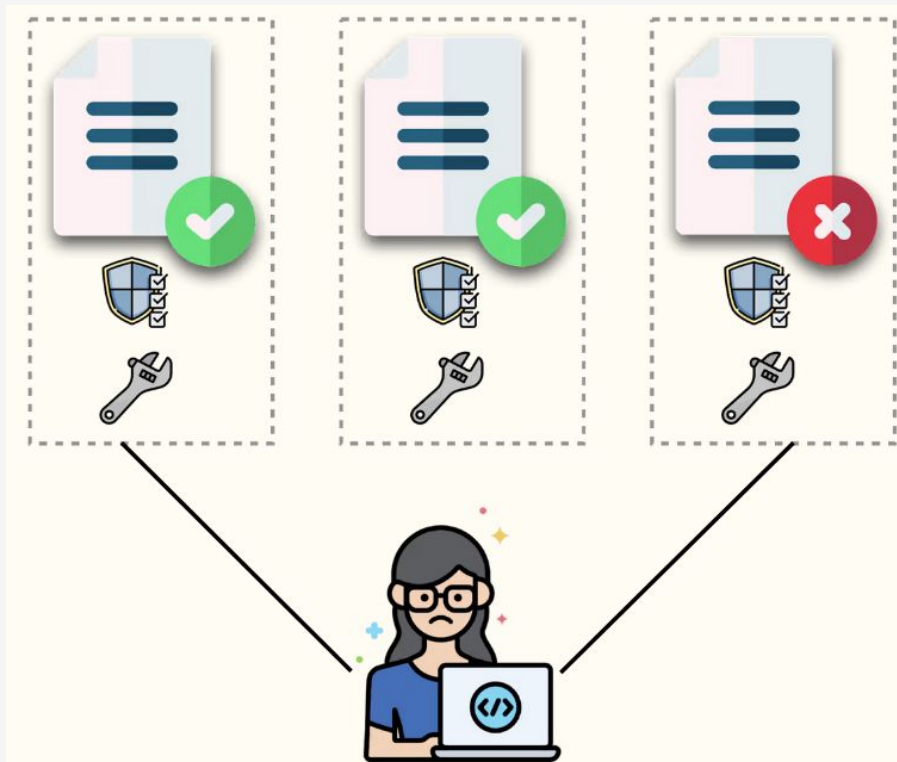


- Các thành phần của hệ thống được **chia nhỏ** thành các phần **độc lập**.
- **Ưu điểm:** phát triển, deploy, cập nhật và mở rộng các thành phần **hoàn toàn độc lập** với nhau.



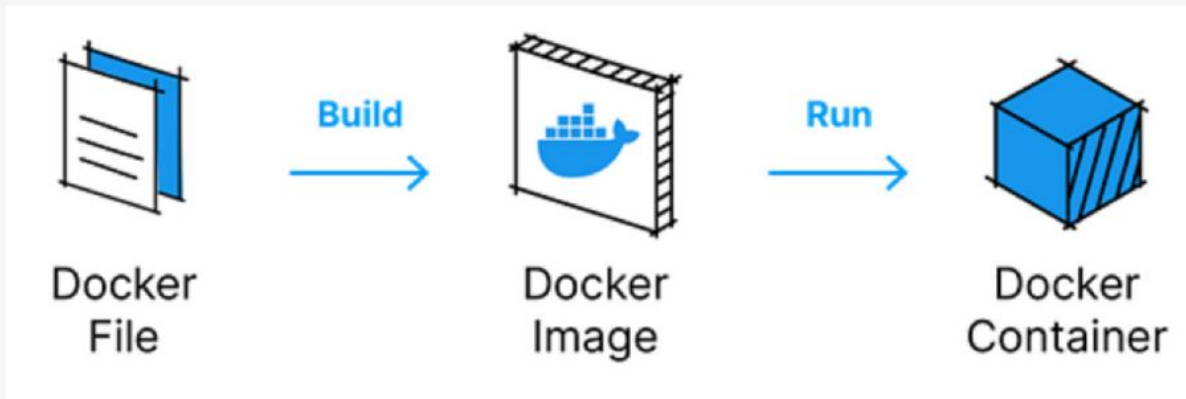
Thách thức Microservices

- Khó khăn trong **triển khai**:
 - giao tiếp giữa các thành phần
 - luồng hoạt động
 - vị trí phần cứng triển khai
- Môi trường phát triển và triển khai **khác nhau**, dễ gây **lỗi** do **không đồng bộ**.
- Nếu sử dụng máy ảo để đảm bảo môi trường thì **yêu cầu tài nguyên lớn**.
- Yêu cầu về một **hệ thống giám sát thường xuyên**.



Docker Container

- **Đóng gói** ứng dụng cùng tất cả phụ thuộc vào một đơn vị **độc lập**
- **Cô lập** ứng dụng khỏi sự khác biệt của **môi trường** cơ sở
- Triết lý "**Build once, run anywhere**"



Khái niệm **chính**



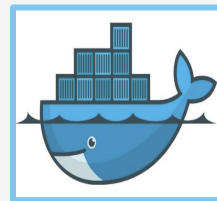
Docker Image
(Ảnh)

Nơi **đóng gói** ứng dụng
và môi trường



Container

Được tạo ra từ Docker
Image, khi chạy sẽ **cô lập**
hoàn toàn với máy chủ
và các tiến trình khác



Registry
(Kho lưu trữ)

Nơi **lưu trữ** các **image**
Docker để chia sẻ

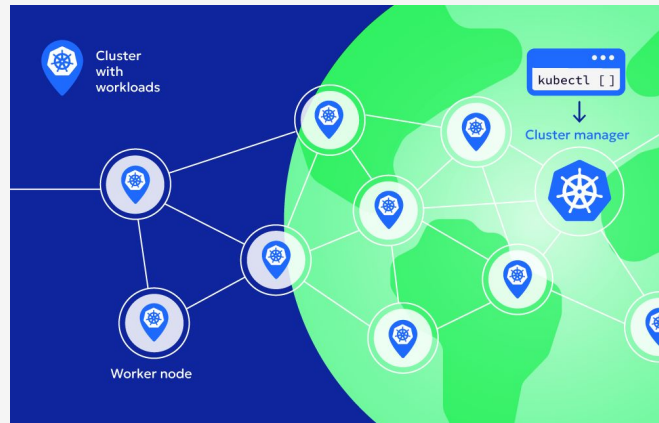
Số lượng 
Container tăng lên

Làm sao để quản lý
chúng hiệu quả 

- Làm sao để duy trì hệ thống **vận hành trơn tru**?
- Deploy các thành phần như nào sao cho **tối ưu tài nguyên, tiết kiệm chi phí** mà vẫn hoạt động **hiệu quả**?

KUBERNETES IS “ALL YOU NEED”

- Hoạt động như **"hệ điều hành"** cho toàn bộ cụm máy chủ
- Cung cấp sẵn các **dịch vụ cơ sở hạ tầng**: Khám phá dịch vụ, mở rộng quy mô tự động, cân bằng tải, tự phục hồi, ...
- Cho phép chạy ứng dụng trên **hàng nghìn node** như thể chúng là một **máy tính duy nhất**
- Giúp các **thành phần** dễ dàng tìm thấy và **giao tiếp** với nhau



Kubernetes: Deploy dễ dàng

- **Pods:** Kubernetes biểu diễn các container thành các Pod để quản lý.

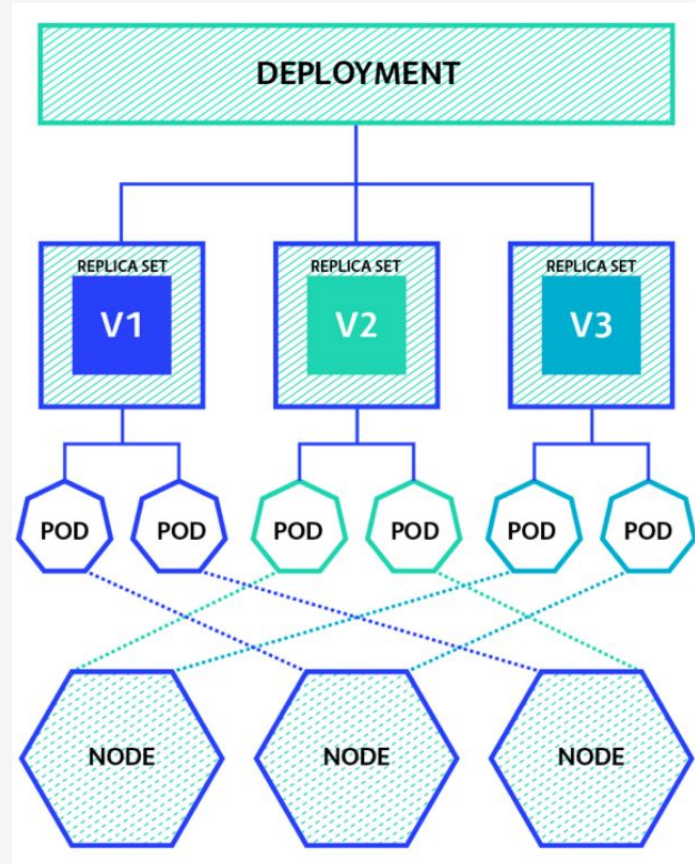
➡ Dễ dàng **triển khai** và **quản lý** các **container**, chia sẻ tài nguyên, tương tác với nhau.

- **Deployments:** Định nghĩa cách ứng dụng sẽ được triển khai, cập nhật và khôi phục.

➡ **Đảm bảo** trạng thái triển khai **khớp** với định nghĩa **mong muốn**.

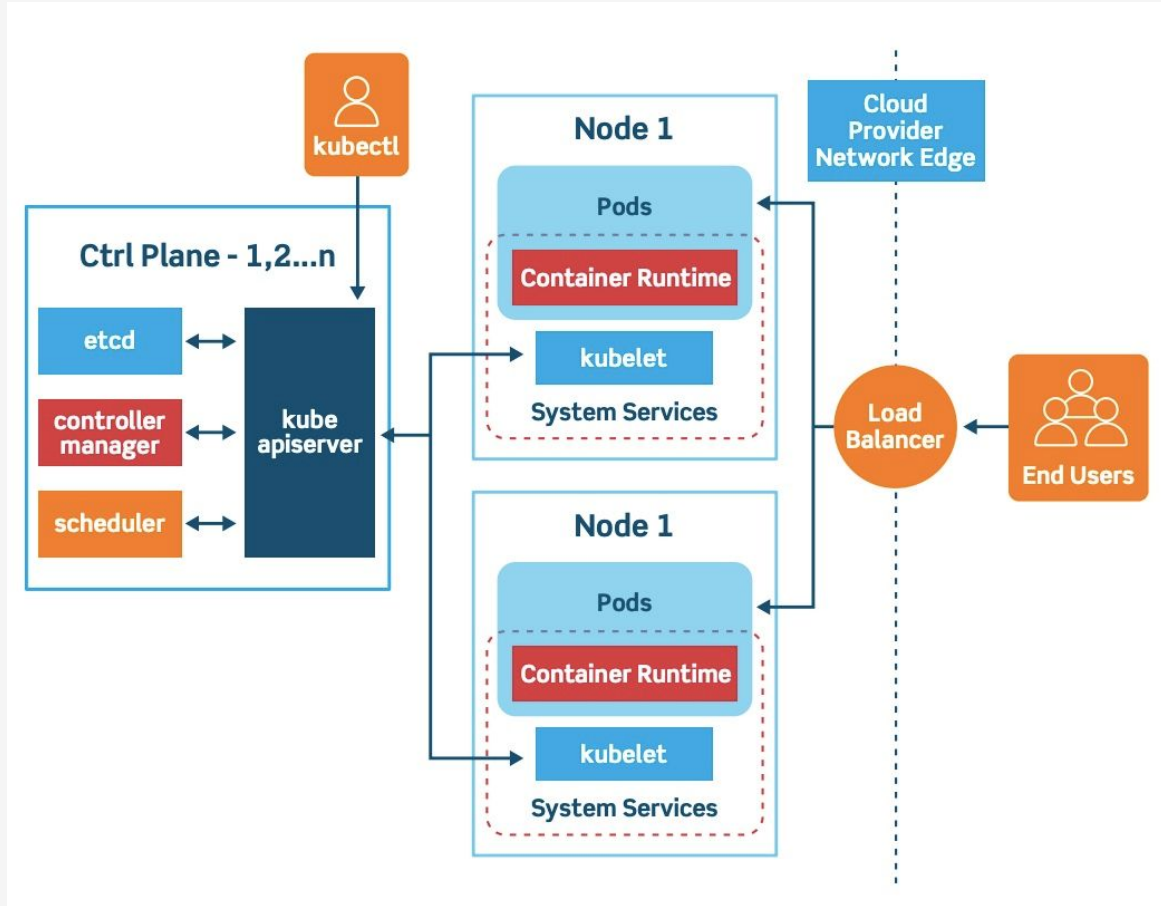
- **ReplicaSets:** Duy trì số lượng **bản sao** container.

➡ **Đảm bảo** ứng dụng luôn hoạt động **ổn định**.



Kiến trúc **Kubernetes** Cluster

- Control Plane
- Worker Nodes





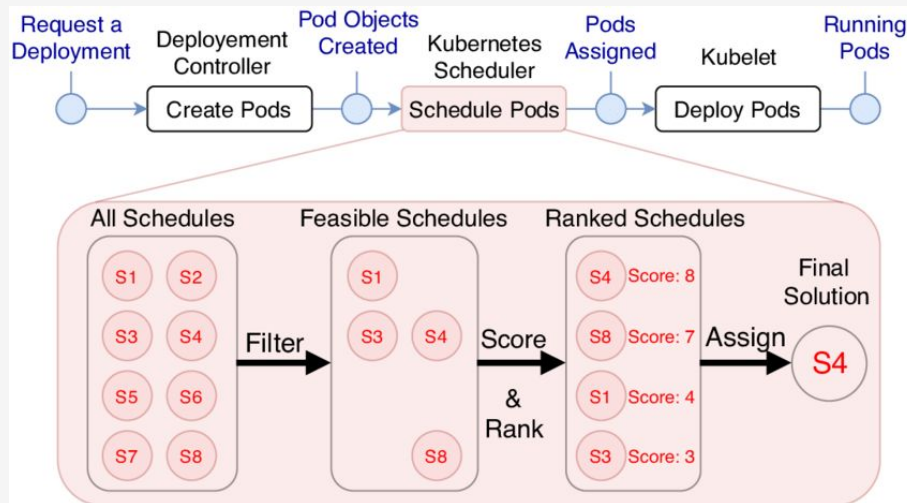
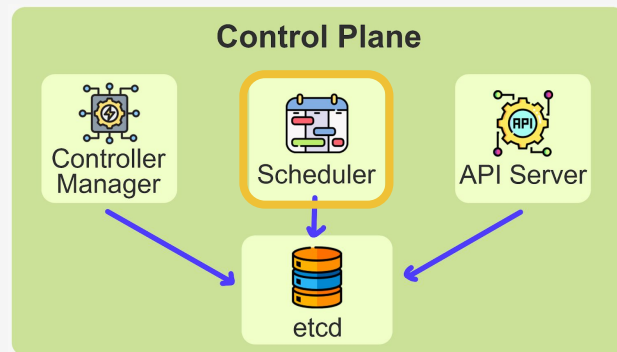
II.

Bộ lập lịch mặc định **kube-scheduler**

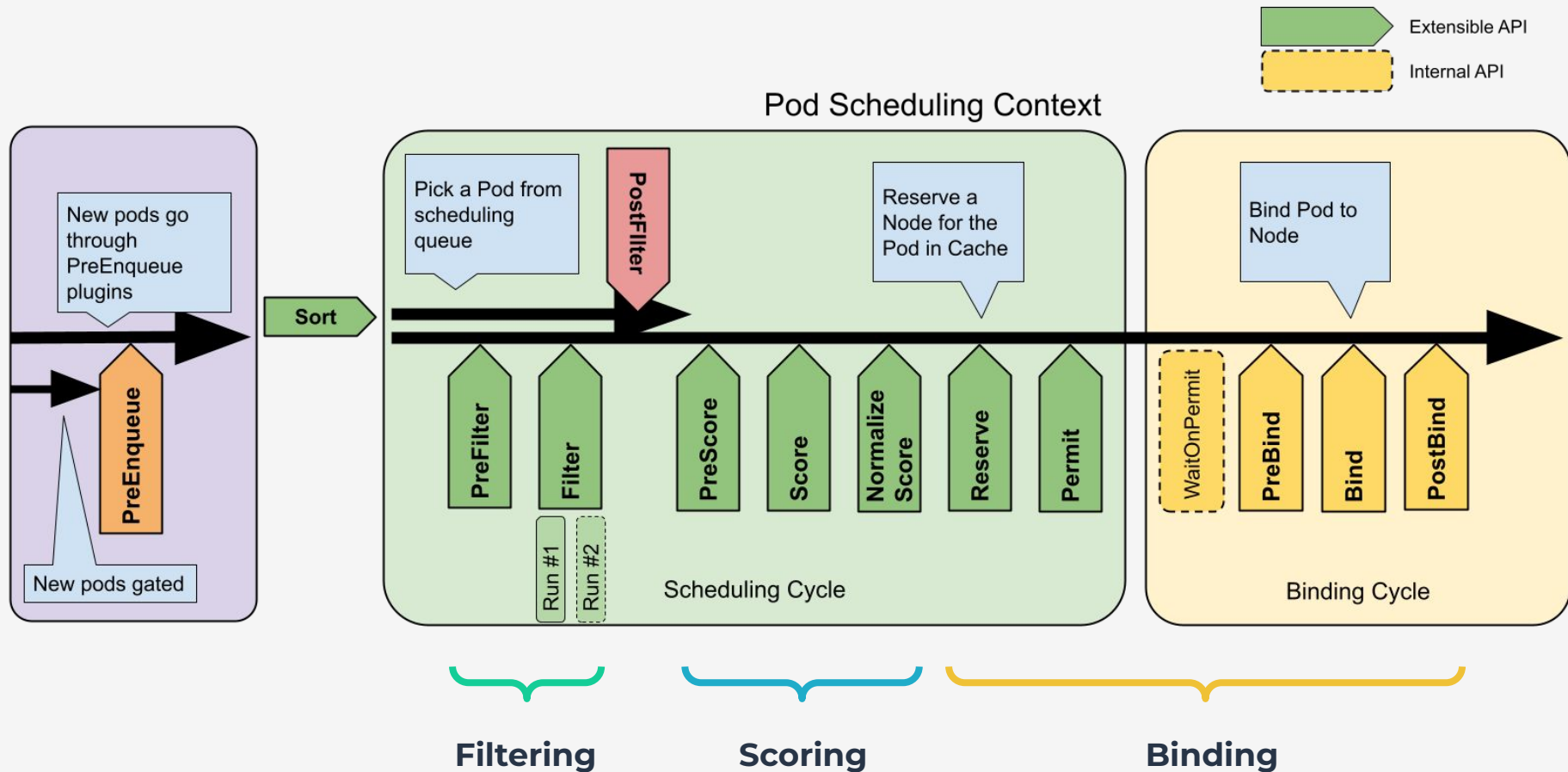


Kubernetes Scheduler

- Thành phần **quan trọng** giúp **gán** các **Pods** đến các **Nodes**.
- Xem xét **tài nguyên** như dung lượng Node (CPU, RAM, GPU), nhãn (labels), taint, và yêu cầu của Pod.
- **Mục tiêu:**
 - **Tối đa hóa** sử dụng **tài nguyên**
 - Đảm bảo **hiệu suất** ứng dụng và tính **sẵn sàng** cao.
 - **Tuân thủ** các **chính sách** bảo mật, phần cứng, và kinh doanh.
- **Mặc định:** **kube-scheduler** (nhưng có thể custom)



Quy trình lập lịch (**kube-scheduler**)



Cơ chế kiểm soát

Node Selector

Pod chỉ chạy trên các **Node** có **nhãn khớp** (ví dụ: **disktype: ssd**)



Taints & Tolerations

Node được "đánh dấu" (**taint**) để **từ chối Pod**; Pod cần "sự chấp nhận" (**toleration**) tương ứng để được lập lịch trên Node bị taint.



Node Affinity/Anti-Affinity

Các **quy tắc** chi tiết hơn (bắt buộc/ưu tiên) để chọn Node dựa trên **nhãn** của **Node** hoặc **nhãn** của các **Pod** khác.



Pod Topology Spread

Phân bố đều các Pod trên các miền (khu vực, vùng, máy chủ) để đảm bảo tính **sẵn sàng cao**.



kube-scheduler đáp
ứng được hầu hết
các yêu cầu 🌟👍

Tại sao cần bộ lập
lịch **tùy chỉnh** ?

- **Tối ưu hóa** cho các nhu cầu **ứng dụng** hoặc **phần cứng cụ thể**.
- Thực thi các **quy tắc** nghiệp vụ **phức tạp** hoặc chính sách bảo mật.
- **Cải thiện** việc **sử dụng tài nguyên** vượt ra ngoài hành vi **mặc định**.



III.

Bộ lập lịch tùy chỉnh custom scheduler



Vấn đề

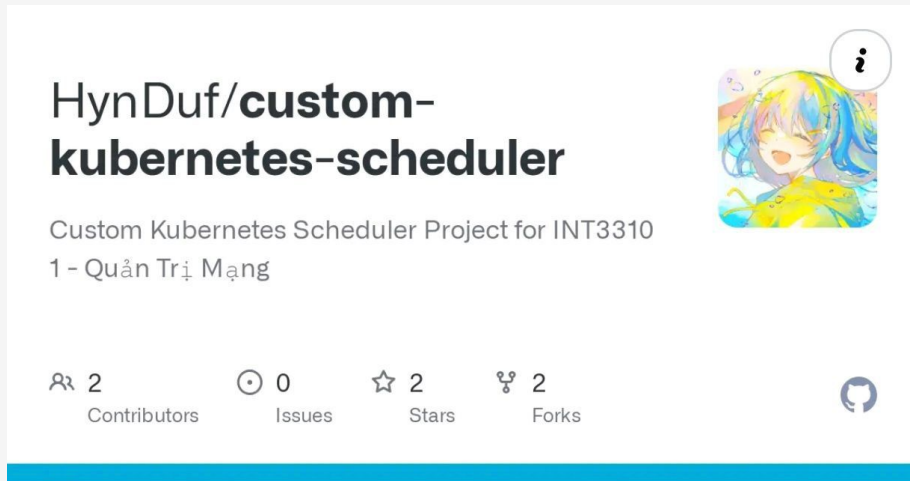
- Chủ yếu dựa vào thông số **requests** và **limits** được **khai báo trước** trong định nghĩa **Pod**
- **Không phát hiện** các node có **tải cao** nhưng **requests thấp**
- Có thể **dẫn đến** tình trạng một số node **quá tải** trong khi các node khác lại **không được tận dụng hiệu quả**

Giải pháp

- Phát triển bộ lập lịch **tùy chỉnh** tích hợp số liệu thời gian thực (**CPU, RAM**) để đặt **Pod** hiệu quả hơn.
- **Tích hợp** các **tính năng** tương tự: Node Selector, Taints & Tolerations, Node Affinity, ...

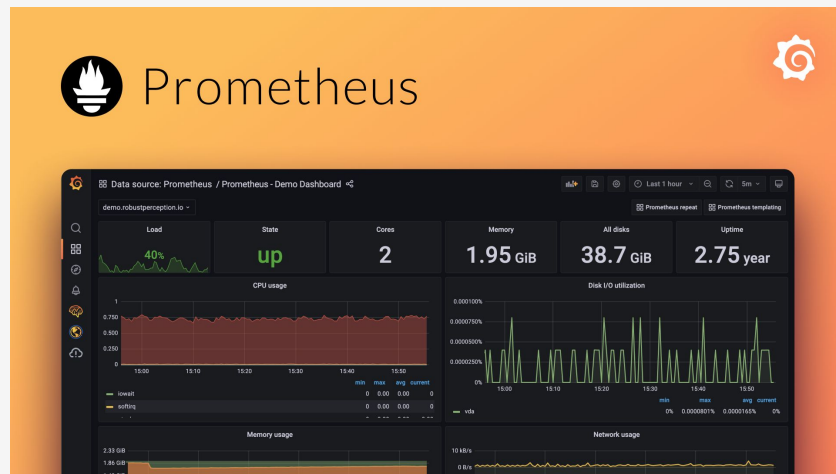
Custom Scheduler

- Cài đặt sử dụng ngôn ngữ **Golang**
- Tương tác với **Kubernetes** qua các **modules** thuộc **k8s.io**
- Mã nguồn tại
<https://github.com/HynDuf/custom-kubernetes-scheduler>

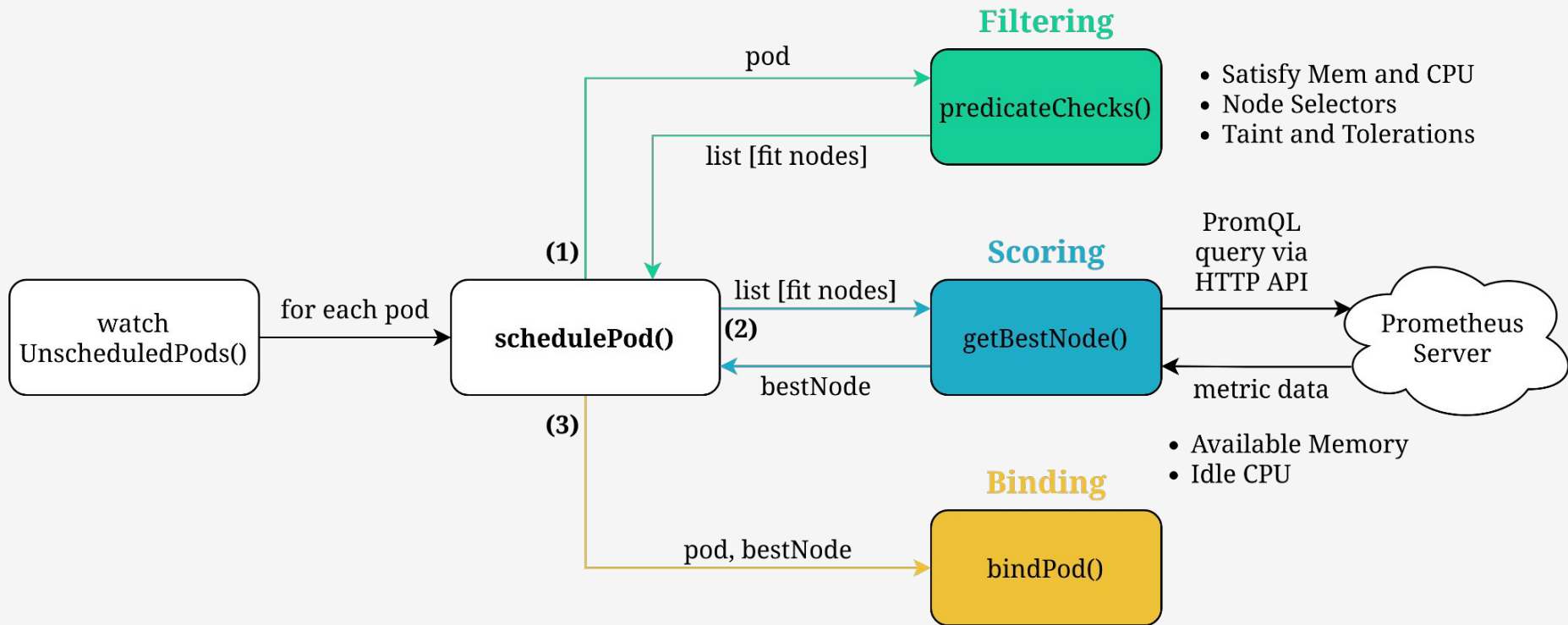


Prometheus

- Một bộ **công cụ mã nguồn mở** hỗ trợ **giám sát** và **cảnh báo**.
- **Thu thập** và **lưu trữ** các **chỉ số** dưới dạng dữ liệu chuỗi thời gian.
- Sử dụng mô hình kéo (**pull model - scraping**) để lấy các **chỉ số** từ các **mục tiêu** được cấu hình (ví dụ: **Node Exporter**).
- **Vai trò trong custom scheduler**: Thu thập dữ liệu hiệu năng **thời gian thực** (**CPU, Bộ nhớ**) từ tất cả các **node** Kubernetes.



Cấu trúc chính



Scoring

- Mem: **node_memory_MemAvailable_bytes**
- CPU: **avg by (instance) (rate(node_cpu_seconds_total{mode="idle"}[1m]))**
- Node Affinity: **preferredDuringSchedulingIgnoredDuringExecution**



- Chuẩn hóa sang thang điểm 10 với mỗi Mem, CPU, Node Affinity:

$$(value / maxValue) * 10$$



Đối với mỗi node, `TotalScore` được tính bằng tổng có trọng số:

```
nodeScore.TotalScore =  
    (config.WeightMem * nodeScore.MemScore)  
    + (config.WeightCPU * nodeScore.CPUScore)  
    + (config.WeightAffinity * nodeScore.AffinityScore);
```





IV.

Demo Custom Scheduler



Demo Cluster

```
g22021121@cloudshell:~/custom-kubernetes-scheduler (valued-night-457210-n0)$ kubectl get pods -o wide -n monitoring
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
node-exporter-9nfg5	1/1	Running	0	14s	10.128.0.13	gke-cluster-2-default-pool-c5dfe2a2-y0v6
node-exporter-lcd8d	1/1	Running	0	14s	10.128.0.12	gke-cluster-2-default-pool-c5dfe2a2-5sif
node-exporter-wkwgx	1/1	Running	0	14s	10.128.0.11	gke-cluster-2-default-pool-c5dfe2a2-kci2
prometheus-deployment-57d8bf4c4-tlpsx	0/1	Running	0	4s	10.44.2.9	gke-cluster-2-default-pool-c5dfe2a2-5sif



5sif

POD

node-exporter

POD

prometheus-deployment



kci2

POD

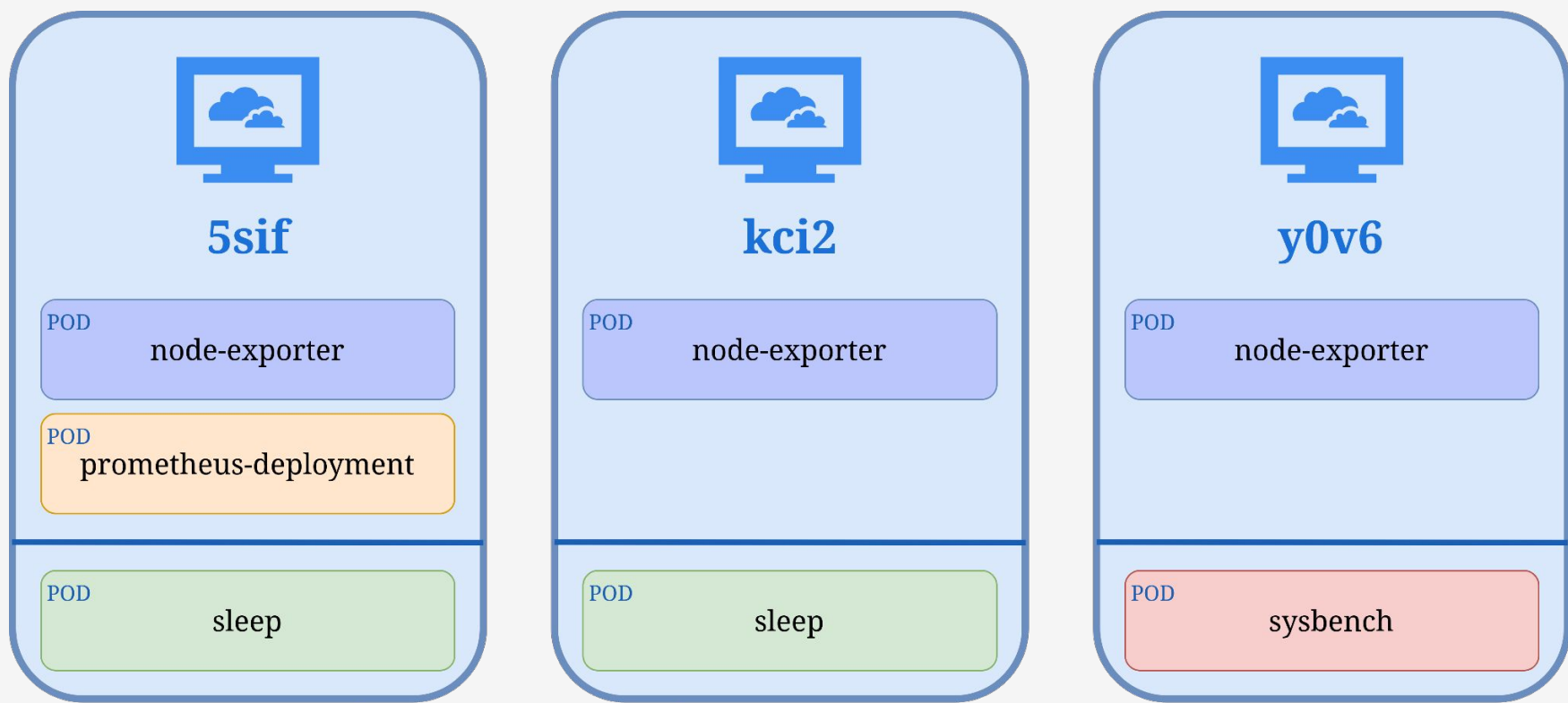
node-exporter



y0v6

POD

node-exporter



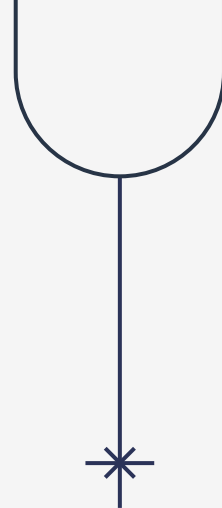
Yêu cầu: 1600 MiB
Nhưng thật sự **không** sử dụng bộ nhớ và CPU

Yêu cầu: 0 MiB
Nhưng thật sự sử dụng cả CPU và bộ nhớ (hơn **2 GiB**)



Xin cảm ơn

Thầy và các bạn đã lắng nghe



Nhóm 1

Huỳnh Tiến Dũng - 21020007

Vũ Huy Hoàng - 22021108

Nguyễn Đăng Quân - 22021121

Nguyễn Hồng Quân - 22021122

Nguyễn Chí Thanh - 22021123

